ASSIGNMENT NO:11.5

Name:Saisharan

Ht.no:2303A51434 bt.no:21

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

class Stack: pass

Expected Output:

• A functional stack implementation with all required methods and docstrings.

```python
"""Task Description #1 - Stack Implementation
Task: Use AI to generate a Stack class with push, pop, peek, and is_empty
methods."""

class Stack:
    """
    A simple implementation of a stack data structure.
    """

    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """
        Add an item to the top of the stack.

        Parameters:
        item: The item to be added to the stack.
        """
        self.items.append(item)

    def pop(self):
        """
        Remove and return the item at the top of the stack.

        Returns:
        The item at the top of the stack.

        Raises:
        IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Pop from an empty stack")
        return self.items.pop()

    def peek(self):
```

```python
class Stack:
    def peek(self):
        """
        Return the item at the top of the stack without removing it.

        Returns:
        The item at the top of the stack.

        Raises:
        IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Peek from an empty stack")
        return self.items[-1]

    def is_empty(self):
        """
        Check if the stack is empty.

        Returns:
        True if the stack is empty, False otherwise.
        """
        return len(self.items) == 0
# Example usage:
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)

    print(stack.peek())  # Output: 3
    print(stack.pop())   # Output: 3
    print(stack.is_empty())  # Output: False
    print(stack.pop())   # Output: 2
    print(stack.pop())   # Output: 1
    print(stack.is_empty())  # Output: True
```

Output:

```
3
3
False
2
1
True
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue: pass

Expected Output: FIFO-based queue class with enqueue, dequeue, peek, and size methods.

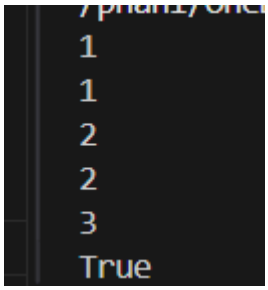
Code screenshots :

```python
"""Task Description #2 - Queue Implementation
Task: Use AI to implement a Queue using Python lists.
Sample Input Code:
class Queue: pass
"""
class Queue:
    """
    A simple implementation of a queue data structure using Python lists.
    """

    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def enqueue(self, item):
        """
        Add an item to the end of the queue.

        Parameters:
        item: The item to be added to the queue.
        """
        self.items.append(item)

    def dequeue(self):
        """
        Remove and return the item at the front of the queue.

        Returns:
        The item at the front of the queue.

        Raises:
        IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("Dequeue from an empty queue")
        return self.items.pop(0)
```

```python
    def peek(self):
        """
        Return the item at the front of the queue without removing it.

        Returns:
        The item at the front of the queue.

        Raises:
        IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("Peek from an empty queue")
        return self.items[0]

    def size(self):
        """
        Return the number of items in the queue.

        Returns:
        The number of items in the queue.
        """
        return len(self.items)

    def is_empty(self):
        """
        Check if the queue is empty.

        Returns:
        True if the queue is empty, False otherwise.
        """
        return len(self.items) == 0
```

```
        return len(self.items) == 0
"""Expected Output:
• FIFO-based queue class with enqueue, dequeue, peek, and size methods.
• Example usage demonstrating the functionality of the queue."""
# Example usage:
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)

    print(queue.peek())  # Output: 1
    print(queue.dequeue())  # Output: 1
    print(queue.size())  # Output: 2
    print(queue.dequeue())  # Output: 2
    print(queue.dequeue())  # Output: 3
    print(queue.is_empty())  # Output: True
```

Output:



Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output: A working linked list implementation with clear method documentation.

Code screenshot :

```
"""Task Description #3 - Linked List
Task: Use AI to generate a Singly Linked List with insert and display
methods.
Sample Input Code:
class Node:
pass
class LinkedList:
pass
Expected Output:
A working linked list implementation with clear method documentation."""
class Node:
    """
    A class representing a node in a singly linked list.

    Attributes:
        data: The value stored in the node.
        next: A reference to the next node in the list.
    """
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    """
    A class representing a singly linked list.

    Attributes:
        head: The first node in the linked list.
    """
    def __init__(self):
        self.head = None

    def insert(self, data):
        """
        Inserts a new node with the given data at the end of the linked list.

        Args:
            data: The value to be stored in the new node.
```

```
            Args:
                data: The value to be stored in the new node.
            """
            new_node = Node(data)
            if self.head is None:
                self.head = new_node
                return
            last_node = self.head
            while last_node.next:
                last_node = last_node.next
            last_node.next = new_node

        def display(self):
            """
            Displays the contents of the linked list.
            """
            current_node = self.head
            while current_node:
                print(current_node.data, end=' ')
                current_node = current_node.next
            print()  # for a new line after displaying the list
# Example usage:
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.insert(10)
    linked_list.insert(20)
    linked_list.insert(30)
    print("Linked List contents:")
    linked_list.display()
```

Output:

```
Linked List contents:
10 20 30
```

Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and

delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output: Collision handling using chaining, with well-commented methods.

Code screenshot:

```python
"""Task Description #4 – Hash Table
Task: Use AI to implement a hash table with basic insert, search, and
delete methods.
Sample Input Code:"""
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """
        Generates a hash value for a given key.
        """
        return hash(key) % self.size

    def insert(self, key, value):
        """
        Inserts a key-value pair into the hash table.
        """
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                return
        bucket.append((key, value))

    def search(self, key):
        """
        Searches for a value associated with a given key in the hash table.
        """
        index = self._hash(key)
        bucket = self.table[index]
        for k, v in bucket:
            if k == key:
                return v
        return None
```

```
    def delete(self, key):
        """
        Deletes a key-value pair from the hash table.
        """
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                del bucket[i]
                return
# Example usage:
hash_table = HashTable()
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
print(hash_table.search("name"))  # Output: Alice
print(hash_table.search("age"))   # Output: 30
hash_table.delete("name")
print(hash_table.search("name"))  # Output: None
```

Output:

```
Alice
30
None
```

Task Description #5 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph: pass

Expected Output:

• Graph with methods to add vertices, add edges, and display

connections.

```python
Task Description #5 – Graph Representation
Task: Use AI to implement a graph using an adjacency list.
Sample Input Code:
class Graph:
    def __init__(self):
        self.adjacency_list = {}

    def add_vertex(self, vertex):
        if vertex not in self.adjacency_list:
            self.adjacency_list[vertex] = []

    def add_edge(self, vertex1, vertex2):
        if vertex1 in self.adjacency_list and vertex2 in self.adjacency_list:
            self.adjacency_list[vertex1].append(vertex2)
            self.adjacency_list[vertex2].append(vertex1)

    def display(self):
        for vertex in self.adjacency_list:
            print(f"{vertex}: {self.adjacency_list[vertex]}")

# Example usage
if __name__ == "__main__":
    graph = Graph()
    graph.add_vertex("A")
    graph.add_vertex("B")
    graph.add_vertex("C")
    graph.add_edge("A", "B")
    graph.add_edge("A", "C")
    graph.add_edge("B", "C")
    graph.display()
```

Output:

```
A: ['B', 'C']
B: ['A', 'C']
C: ['A', 'B']
```

Task Description #6: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.

2. Emergency Case Handling – Critical patients must be treated first.

3. Medical Records Storage – Fast retrieval of patient details using ID.

4. Doctor Appointment Scheduling – Appointments sorted by time.

5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

• For each feature, select the most appropriate data structure from the list below:

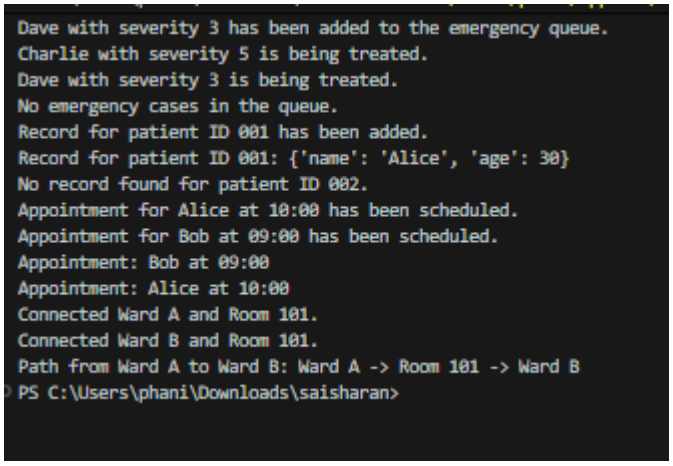
o Stack	o Queue
o Priority Queue	o Linked List
o Binary Search Tree (BST)	o Graph
o Hash Table	o Deque

code screenshot:

```python
class PatientCheckInSystem:
    def __init__(self):
        self.queue = []

    def check_in(self, patient_name):
        self.queue.append(patient_name)
        print(f"{patient_name} has checked in.")

    def next_patient(self):
        if self.queue:
            patient = self.queue.pop(0)
            print(f"{patient} is being treated.")
        else:
            print("No patients in the queue.")
# 2. Emergency Case Handling using Priority Queue
import heapq
class EmergencyCaseHandling:
    def __init__(self):
        self.priority_queue = []

    def add_case(self, patient_name, severity):
        heapq.heappush(self.priority_queue, (-severity, patient_name))
        print(f"{patient_name} with severity {severity} has been added to the emergency queue.")

    def next_case(self):
        if self.priority_queue:
            severity, patient = heapq.heappop(self.priority_queue)
            print(f"{patient} with severity {-severity} is being treated.")
        else:
            print("No emergency cases in the queue.")
# 3. Medical Records Storage using Hash Table
class MedicalRecordsStorage:

    def __init__(self):
        self.records = {}

    def add_record(self, patient_id, details):
        self.records[patient_id] = details
        print(f"Record for patient ID {patient_id} has been added.")

    def get_record(self, patient_id):
        record = self.records.get(patient_id)
        if record:
            print(f"Record for patient ID {patient_id}: {record}")
        else:
            print(f"No record found for patient ID {patient_id}.")
# 4. Doctor Appointment Scheduling using Linked List
class Appointment
    def __init__(self, time, patient_name):
        self.time = time
        self.patient_name = patient_name
        self.next = None
class DoctorAppointmentScheduling:
    def __init__(self):
        self.head = None

    def schedule_appointment(self, time, patient_name):
        new_appointment = Appointment(time, patient_name)
        if not self.head or self.head.time > time:
            new_appointment.next = self.head
            self.head = new_appointment
        else:
            current = self.head
            while current.next and current.next.time < time:
                current = current.next
            new_appointment.next = current.next
            current.next = new appointment
```

```
class DoctorAppointmentScheduling:
    def schedule_appointment(self, time, patient_name):
        print(f"Appointment for {patient_name} at {time} has been scheduled.")

    def print_appointments(self):
        current = self.head
        while current:
            print(f"Appointment: {current.patient_name} at {current.time}")
            current = current.next
# 5. Hospital Room Navigation using Graph
class Graph:
    def __init__(self):
        self.graph = {}

    def add_room(self, room):
        if room not in self.graph:
            self.graph[room] = []

    def connect_rooms(self, room1, room2):
        if room1 in self.graph and room2 in self.graph:
            self.graph[room1].append(room2)
            self.graph[room2].append(room1)
            print(f"Connected {room1} and {room2}.")
        else:
            print("One or both rooms do not exist.")

    def navigate(self, start_room, end_room):
        visited = set()
        queue = [(start_room, [start_room])]
        while queue:
            current_room, path = queue.pop(0)
            if current_room == end_room:
                print(f"Path from {start_room} to {end_room}: {' -> '.join(path)}")
                return
            if current_room not in visited:
                visited.add(current_room)
                for neighbor in self.graph.get(current_room, []):
                    queue.append((neighbor, path + [neighbor]))
        print(f"No path found from {start_room} to {end_room}.")
# Example usage
if __name__ == "__main__":
    # Patient Check-In System
    check_in_system = PatientCheckInSystem()
    check_in_system.check_in("Alice")
    check_in_system.check_in("Bob")
    check_in_system.next_patient()
    check_in_system.next_patient()
    check_in_system.next_patient()

    # Emergency Case Handling
    emergency_handling = EmergencyCaseHandling()
    emergency_handling.add_case("Charlie", 5)
    emergency_handling.add_case("Dave", 3)
    emergency_handling.next_case()
    emergency_handling.next_case()
    emergency_handling.next_case()

    # Medical Records Storage
    records_storage = MedicalRecordsStorage()
    records_storage.add_record("001", {"name": "Alice", "age": 30})
    records_storage.get_record("001")
    records_storage.get_record("002")

    # Doctor Appointment Scheduling
    appointment_scheduling = DoctorAppointmentScheduling()
    appointment_scheduling.schedule_appointment("10:00", "Alice")
    appointment_scheduling.schedule_appointment("09:00", "Bob")
```

Output:



1. Patient Check-In System – Queue

2. Emergency Case Handling – Priority Queue

3. Medical Records Storage – Hash Table

4. Doctor Appointment Scheduling – Linked List

5. Hospital Room Navigation – Graph

Task Description #7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

• For each feature, select the most appropriate data structure from

the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Code screenshot:

```
# Feature → Chosen Data Structure → Justification

| Feature                          | Chosen Data Structure | Justification                                                                          |
|----------------------------------|-----------------------|----------------------------------------------------------------------------------------|
| Traffic Signal Queue             | Queue                 | A queue is ideal for managing vehicles waiting at traffic
| Emergency Vehicle Priority Handling | Priority Queue        | A priority queue allows for prioritizing emergency vehicles over regular traffic. |
| Vehicle Registration Lookup      | Hash Table            | A hash table provides fast access to vehicle details based on registration numbers. |
| Road Network Mapping             | Graph                 | A graph can represent roads and intersections effectively, allowing for complex relationships. |
| Parking Slot Availability        | Linked List           | A linked list can efficiently manage the dynamic nature of parking slot availability. |'''
# Implementing the Emergency Vehicle Priority Handling feature using a Priority Queue
import heapq
class EmergencyVehiclePriorityQueue:
    def __init__(self):
        """Initialize an empty priority queue."""
        self.queue = []

    def add_vehicle(self, vehicle_type, vehicle_id):
        """
        Add a vehicle to the priority queue.

        Parameters:
        vehicle_type (str): The type of the vehicle (e.g., 'ambulance', 'fire_truck', 'car').
        vehicle_id (str): The unique identifier for the vehicle.
        """
        priority = self.get_priority(vehicle_type)
        heapq.heappush(self.queue, (priority, vehicle_id))

    def get_priority(self, vehicle_type):
        """Assign priority based on vehicle type."""
        if vehicle_type == 'ambulance':
            return 1  # Highest priority
        elif vehicle_type == 'fire_truck':
            return 2  # Second highest priority
        else:
            return 3  # Regular vehicles

    def dispatch_vehicle(self):
        """
        Dispatch the highest priority vehicle from the queue.

        Returns:
        str: The ID of the dispatched vehicle or None if the queue is empty.
        """
        if self.queue:
            return heapq.heappop(self.queue)[1]  # Return the vehicle ID
        return None
# Example usage
if __name__ == "__main__":
    priority_queue = EmergencyVehiclePriorityQueue()
    priority_queue.add_vehicle('car', 'CAR123')
    priority_queue.add_vehicle('ambulance', 'AMB456')
    priority_queue.add_vehicle('fire_truck', 'FIRE789')

    print("Dispatching vehicles in order of priority:")
    print(priority_queue.dispatch_vehicle())  # Should dispatch AMB456 first
    print(priority_queue.dispatch_vehicle())  # Should dispatch FIRE789 next
    print(priority_queue.dispatch_vehicle())  # Should dispatch CAR123 last
    print(priority_queue.dispatch_vehicle())  # Should return None (queue is empty)
```

Output:

```
Dispatching vehicles in order of priority:
AMB456
FIRE789
CAR123
None
```

Task Description #8: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.

2. Order Processing System – Orders processed in the order they are placed.

3. Top-Selling Products Tracker – Products ranked by sales count.

4. Product Search Engine – Fast lookup of products using product ID.

5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack o Queue

o Priority Queue o Linked List

o Binary Search Tree (BST) o Graph

o Hash Table o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Code screenshot:

```
# Feature to Data Structure Mapping
| Feature                        | Chosen Data Structure | Justification                                                                                    |
|--------------------------------|-----------------------|--------------------------------------------------------------------------------------------------|
| Shopping Cart Management       | Linked List           | A linked list allows for dynamic addition and removal of products without needing to shift elements, making it efficient for managing a shopping cart. |
| Order Processing System        | Queue                 | A queue follows the First-In-First-Out (FIFO) principle, which is ideal for processing orders in the order they are placed. |
| Top-Selling Products Tracker    | Priority Queue        | A priority queue can efficiently keep track of products based on their sales count, allowing for quick retrieval of top-selling products. |
| Product Search Engine          | Hash Table            | A hash table provides constant average time complexity for lookups, making it ideal for fast retrieval of products using their IDs. |
| Delivery Route Planning        | Graph                 | A graph can represent the connections between warehouses and delivery locations, allowing for efficient route planning using algorithms like Dijkstra's or A*. |
# Python program implementing the Product Search Engine using a Hash Table
class ProductSearchEngine:
    """
    A simple product search engine using a hash table (dictionary) for fast lookups.
    """

    def __init__(self):
        """Initialize the product search engine with an empty hash table."""
        self.products = {}

    def add_product(self, product_id, product_info):
        """
        Add a product to the search engine.

        :param product_id: Unique identifier for the product
        :param product_info: Dictionary containing product details (e.g., name, price)
        """
        self.products[product_id] = product_info

    def search_product(self, product_id):
        """
        Search for a product by its ID.

        :param product_id: Unique identifier for the product
        :return: Product information if found, else None
        """
        return self.products.get(product_id, None)
# Example usage
if __name__ == "__main__":
    search_engine = ProductSearchEngine()
    search_engine.add_product("P001", {"name": "Laptop", "price": 999.99})
    search_engine.add_product("P002", {"name": "Smartphone", "price": 499.99})

    product = search_engine.search_product("P001")
    if product:
        print(f"Product found: {product}")
    else:
        print("Product not found.")
    product = search_engine.search_product("P003")
    if product:
        print(f"Product found: {product}")
    else:
        print("Product not found.")
```

Ouput: